Le microservice Study Community

Projet CampusFind, ESPRIT — Document de conception architecturale et fonctionnelle

Le microservice Study Community permet aux étudiants de créer des groupes de révision, de rejoindre des groupes, de programmer des séances et d'échanger des messages. Il possède sa propre base de données PostgreSQL. Il ne gère ni les comptes utilisateurs, ni les réservations réelles de salles, ni l'envoi des notifications.

1. Conception du microservice : diagramme de classes

Le service comporte six entités métier : Matiere, GroupeEtude, AdhesionGroupe, SeanceRevision, ParticipationSeance et MessageGroupe. Les énumérations sont placées à côté des classes concernées. Chaque classe possède un identifiant id de type Long.

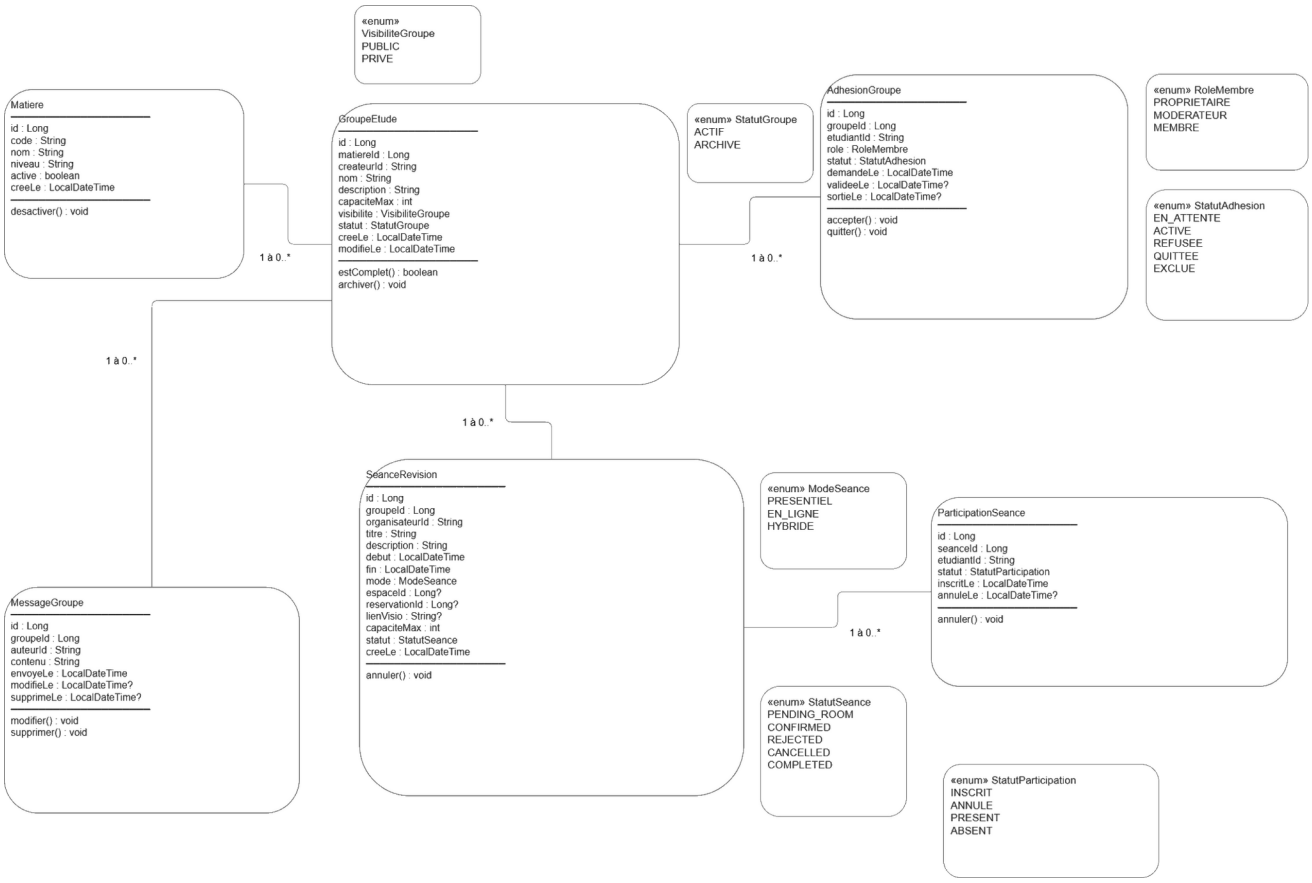


Figure 1 — Diagramme UML de Study Community Service (les identifiants des utilisateurs proviennent de Keycloak).

Les champs espaceId et reservationId sont de simples références vers Room Reservation ; aucune table de sa base n'est partagée avec Study Community.

2. Interactions avec les autres microservices

Study Community communique principalement avec Room Reservation pour réserver une salle et avec Notification Service pour informer les étudiants. Ces deux communications sont différentes :

Service	Mode	Données échangées / objectif
Room Reservation	Synchrone – REST / OpenFeign	Envoyer espaceId, sessionId, groupId, début, fin et nombre de participants ; récupérer reservationId.
Notification Service	Asynchrone – RabbitMQ	Publier NewGroupActivity après confirmation ou annulation d'une activité du groupe.
User Management	Pas d'appel obligatoire	Les identifiants étudiants sont obtenus depuis le JWT Keycloak.

2.1 Communication synchrone : réservation d'une salle

Exemple : un responsable organise une séance de Big Data pour 8 personnes dans une salle du campus.

1. Study Community vérifie que l'organisateur appartient au groupe et a le droit de créer une séance.

2. Il enregistre une séance provisoire avec le statut PENDING_ROOM et un identifiant sessionId.

3. Avec OpenFeign, il appelle POST /api/reservations de Room Reservation en envoyant le créneau et la salle.

4. Room Reservation vérifie la disponibilité, les horaires, la capacité et les conflits de réservation.

5. En cas de succès (201), il retourne reservationId ; Study Community le conserve et passe la séance à CONFIRMED.

6. Si la séance est annulée, Study Community appelle PATCH /api/reservations/{id}/cancel, puis finalise l'annulation locale.

Gestion des erreurs et des pannes

Réponse Room Reservation	Réaction de Study Community
201 Created / 200 OK	Réservation créée ou déjà existante (rejeu idempotent) ; confirmer la séance.
409 Conflict	Créneau occupé ou identifiant déjà utilisé ; refuser la réservation et expliquer la cause.
422 Unprocessable	Capacité ou horaires non respectés ; marquer la demande REJECTED.
503 / timeout	Ne pas confirmer la séance ; conserver PENDING_ROOM puis vérifier/réessayer avec le même sessionId.

Important : en cas de timeout, Room Reservation peut avoir créé la réservation sans que sa réponse arrive. C'est pourquoi le même sessionId doit permettre de retrouver ou rejouer la demande sans doublon.

2.2 Vue d’ensemble des interactions

Le tableau suivant présente le trajet des demandes et des événements. Les appels REST nécessitent une réponse immédiate, contrairement aux notifications envoyées via RabbitMQ.

Source	Destination	Action	Type
Frontend	API Gateway	Envoie la demande de séance avec le JWT.	REST (synchrone)
API Gateway	Study Community	Vérifie et transmet la requête au bon service.	REST (synchrone)
Study Community	Room Reservation	Demande de réserver une salle : POST /api/reservations.	OpenFeign (synchrone)
Room Reservation	Study Community	Renvoie 201 + reservationId, ou 409 / 422 / 503 en cas d’erreur.	REST (synchrone)
Study Community	PostgreSQL community_db	Enregistre le groupe, la séance et reservationId.	Accès à sa base
Room Reservation	PostgreSQL reservation_db	Enregistre la réservation de salle.	Accès à sa base
Study Community	RabbitMQ	Publie NewGroupActivity après confirmation de la séance.	Asynchrone
Room Reservation	RabbitMQ	Publie ReservationConfirmed ou ReservationCancelled.	Asynchrone
RabbitMQ	Notification Service	Transmet les événements aux files de notifications.	Asynchrone
Keycloak / Eureka	Services concernés	Fournissent le JWT et la découverte des services.	Infrastructure

Remarque : Study Community et Room Reservation possèdent chacun leur base PostgreSQL. Le service de notifications ne bloque pas une réservation déjà confirmée.

2.3 Communication asynchrone : RabbitMQ

Après la confirmation de la séance, Study Community publie un événement NewGroupActivity dans l’exchange community.events (clé community.activity.new). Notification Service reçoit cet événement et avertit les membres du groupe. Il ne faut pas bloquer la réservation si la notification est temporairement indisponible.

De son côté, Room Reservation publie ReservationConfirmed ou ReservationCancelled dans reservation.events pour les notifications liées directement à la réservation. Les destinataires doivent être définis pour éviter une double notification.

Événement	Produit par	Utilité
NewGroupActivity	Study Community	Informer les membres d’une nouvelle séance confirmée.
GroupSessionCancelled	Study Community	Informer les membres de l’annulation de la séance (événement proposé).
ReservationConfirmed	Room Reservation	Informer le réservataire que la salle est confirmée.
ReservationCancelled	Room Reservation	Informer le réservataire de l’annulation de la salle.

3. Contrats d’API — Study Community

Les endpoints suivants sont proposés pour le microservice. Les requêtes passent par l’API Gateway ; les opérations sensibles nécessitent un JWT valide et des droits adaptés.

Méthode	Endpoint	Fonction	Succès
GET	/api/subjects	Lister les matières	200
GET	/api/groups	Chercher des groupes	200
POST	/api/groups	Créer un groupe	201
GET	/api/groups/{id}	Consulter un groupe	200
PATCH	/api/groups/{id}	Modifier un groupe	200
POST	/api/groups/{id}/memberships	Rejoindre / demander l’accès	201 / 202
PATCH	/api/groups/{id}/memberships/{id}/approve	Accepter une demande	200
POST	/api/groups/{id}/sessions	Créer une séance	201 / 202
GET	/api/groups/{id}/sessions	Lister les séances	200
PATCH	/api/sessions/{id}/cancel	Annuler une séance	200
POST	/api/sessions/{id}/participations	S’inscrire à une séance	201
GET	/api/groups/{id}/messages	Lire la discussion	200
POST	/api/groups/{id}/messages	Publier un message	201

4. Interactions avec l’infrastructure

Composant	Rôle
API Gateway	Point d’entrée du frontend, transmet la requête au bon microservice.
Keycloak	Authentification ; émet le JWT contenant l’identifiant et les rôles de l’utilisateur.
Eureka	Permet à OpenFeign de localiser Room Reservation grâce au nom du service.
Config Server	Centralise les paramètres techniques des applications, si activé dans l’équipe.
Docker Compose	Démarre les microservices, bases et infrastructure dans des conteneurs.

5. Choix de PostgreSQL

Study Community utilise une base indépendante appelée community_db. PostgreSQL est adapté car les groupes, les membres, les séances et les inscriptions possèdent plusieurs relations. Ses transactions permettent de vérifier les règles métier avant l’enregistrement, par exemple éviter qu’un membre soit inscrit deux fois à la même séance.

- Une base par service : Room Reservation possède une autre base, sans accès direct depuis Study Community.
- Contraintes de données : UNIQUE(groupe_id, etudiant_id) et UNIQUE(seance_id, etudiant_id).
- Références externes : les identifiants Keycloak et reservationId ne sont pas des clés étrangères entre microservices.
- Types d’identifiants : id Long pour les entités ; identifiants utilisateur String pour le claim sub de Keycloak.

6. Énumérations du diagramme de classes

Énumération	Valeurs
VisibiliteGroupe	PUBLIC, PRIVE
StatutGroupe	ACTIF, ARCHIVE
RoleMembre	PROPRIETAIRE, MODERATEUR, MEMBRE
StatutAdhesion	EN_ATTENTE, ACTIVE, REFUSEE, QUITTEE, EXCLUE
ModeSeance	PRESENTIEL, EN_LIGNE, HYBRIDE
StatutSeance	PENDING_ROOM, CONFIRMED, REJECTED, CANCELLED, COMPLETED
StatutParticipation	INSCRIT, ANNULE, PRESENT, ABSENT

7. Règles de fonctionnement

- Un créateur de groupe devient PROPRIETAIRE et membre actif. Un groupe PRIVE exige une validation.
- Seul un membre actif peut participer à une séance ou publier un message ; seul un responsable autorisé peut organiser une séance.
- Une séance en présentiel ou hybride nécessitant une salle du campus passe par Room Reservation ; une séance EN_LIGNE peut être confirmée sans réservation physique.
- Un événement de notification n’est publié qu’après confirmation effective de l’activité. Les consommateurs traitent eventId pour éviter les doublons.